Map Colouring Problem

1. Aim

To implement the **Map Colouring problem** using a constraint satisfaction and backtracking approach to assign colours to different regions of a map such that no two adjacent regions have the same colour.

2. Problem Statement

The Map Colouring problem is a classic Artificial Intelligence problem in which different regions of a map must be assigned colours so that **no two neighbouring regions share the same colour**. The objective is to find a valid colouring using a limited number of colours while satisfying all the given constraints.

3. Solution

The problem is solved using the **backtracking technique**. A colour is assigned to each region one at a time, and the assignment is checked against the colours of its neighbouring regions. If a conflict occurs, the algorithm backtracks and tries a different colour. This process continues until all regions are assigned valid colours.

4. Algorithm

1. Start with a map containing different regions and their neighbouring regions.

2. Define the available colours.
3. Select an uncoloured region.
4. Assign one of the available colours to the selected region.
5. Check whether the assigned colour conflicts with any neighbouring region.
6. If there is no conflict, move to the next region.
7. If a conflict occurs, try another available colour.
8. If no colour is possible, backtrack to the previous region.
9. Continue until all regions are successfully coloured.
10. Display the final colour assignment.

5. Program

```
def is_safe(region, color, assignment, graph):
    for neighbor in graph[region]:
        if assignment.get(neighbor) == color:
            return False
    return True



def map_coloring(graph, colors, assignment, regions):
```

```
    if len(assignment) == len(regions):
        return True

    region = regions[len(assignment)]

    for color in colors:
        if is_safe(region, color, assignment, graph):
            assignment[region] = color

            if map_coloring(graph, colors, assignment,
regions):
                return True

            del assignment[region]

    return False



# Map regions and their neighbours
graph = {
    "A": ["B", "C"],
```

```
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

# Available colours
colors = ["Red", "Green", "Blue"]

assignment = {}
regions = list(graph.keys())

if map_coloring(graph, colors, assignment, regions):
    print("Map Coloring Solution:")

    for region in regions:
        print(region, "->", assignment[region])
else:
    print("No solution exists.")
```

6. Result

The Map Colouring problem was successfully implemented using the **backtracking algorithm**. The program assigned colours to all the regions while

ensuring that no two adjacent regions have the same colour.

Output:

Map Coloring Solution:

A -> Red

B -> Green

C -> Blue

D -> Red

Thus, the **Map Colouring problem was successfully solved using three colours**, satisfying the constraint that adjacent regions must have different colours.